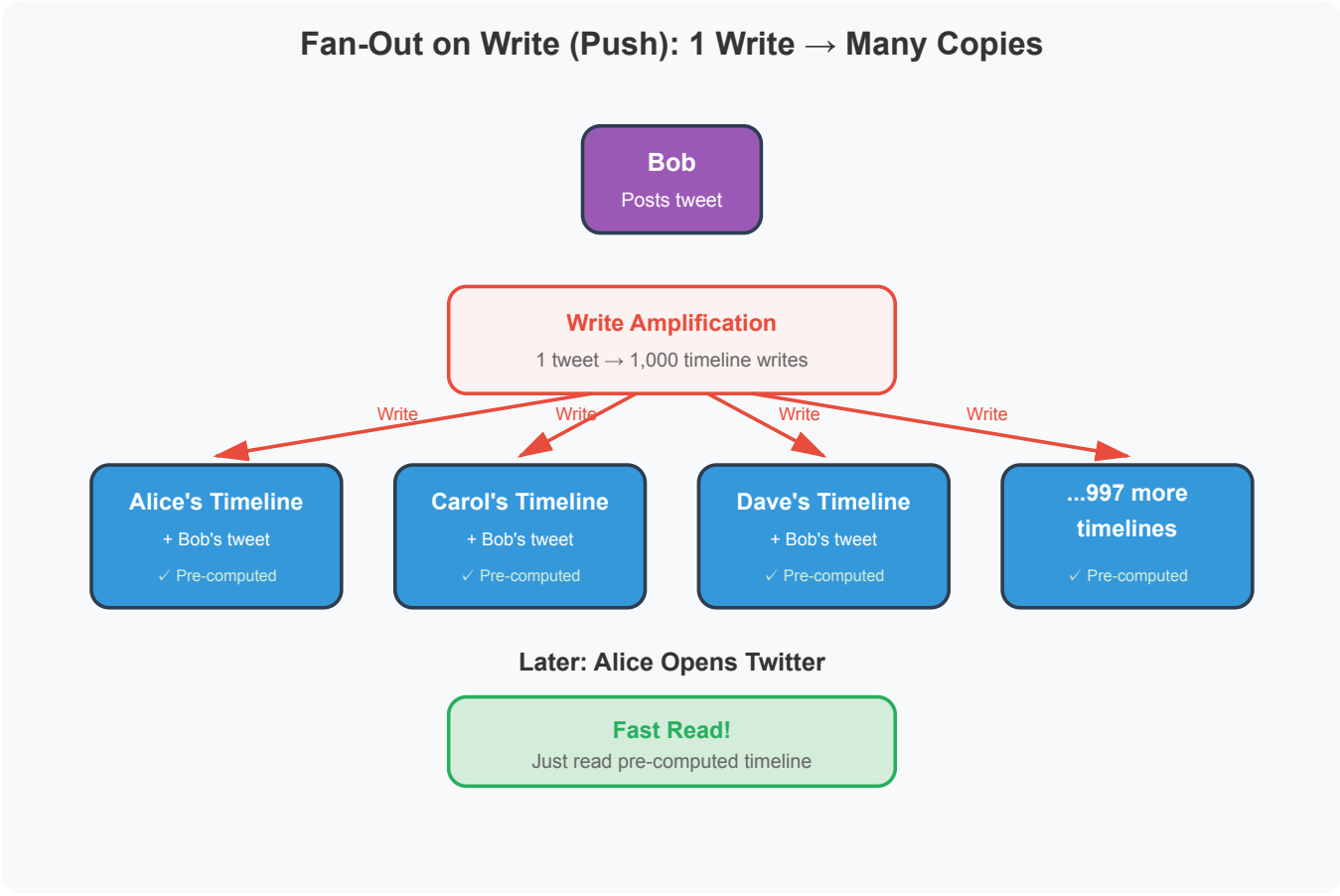
Push vs Pull in Twitter's Timeline System Design

systemdesignschool.io/fundamentals/push-vs-pull-twitter-timeline



Push vs Pull in Twitter Timeline

Push delivers real-time updates but struggles with scale. Pull handles millions of clients but adds latency. Twitter timelines need both: immediate updates for active users and efficient resource usage for 300 million users. How does Twitter decide when to push and when to pull?

The Timeline Problem

A user opens Twitter. Their timeline shows tweets from everyone they follow. Alice follows 500 accounts. Some post frequently. Others post once a week. When Alice loads her timeline, the system must show recent tweets from all 500 accounts, sorted by time.

The simple approach queries on demand. Fetch tweets from each followed account. Merge and sort them. Return the results. This is pull. It works for small follower counts but breaks at scale. Querying 500 accounts, sorting thousands of tweets, and merging results takes time. Every timeline load repeats this work.

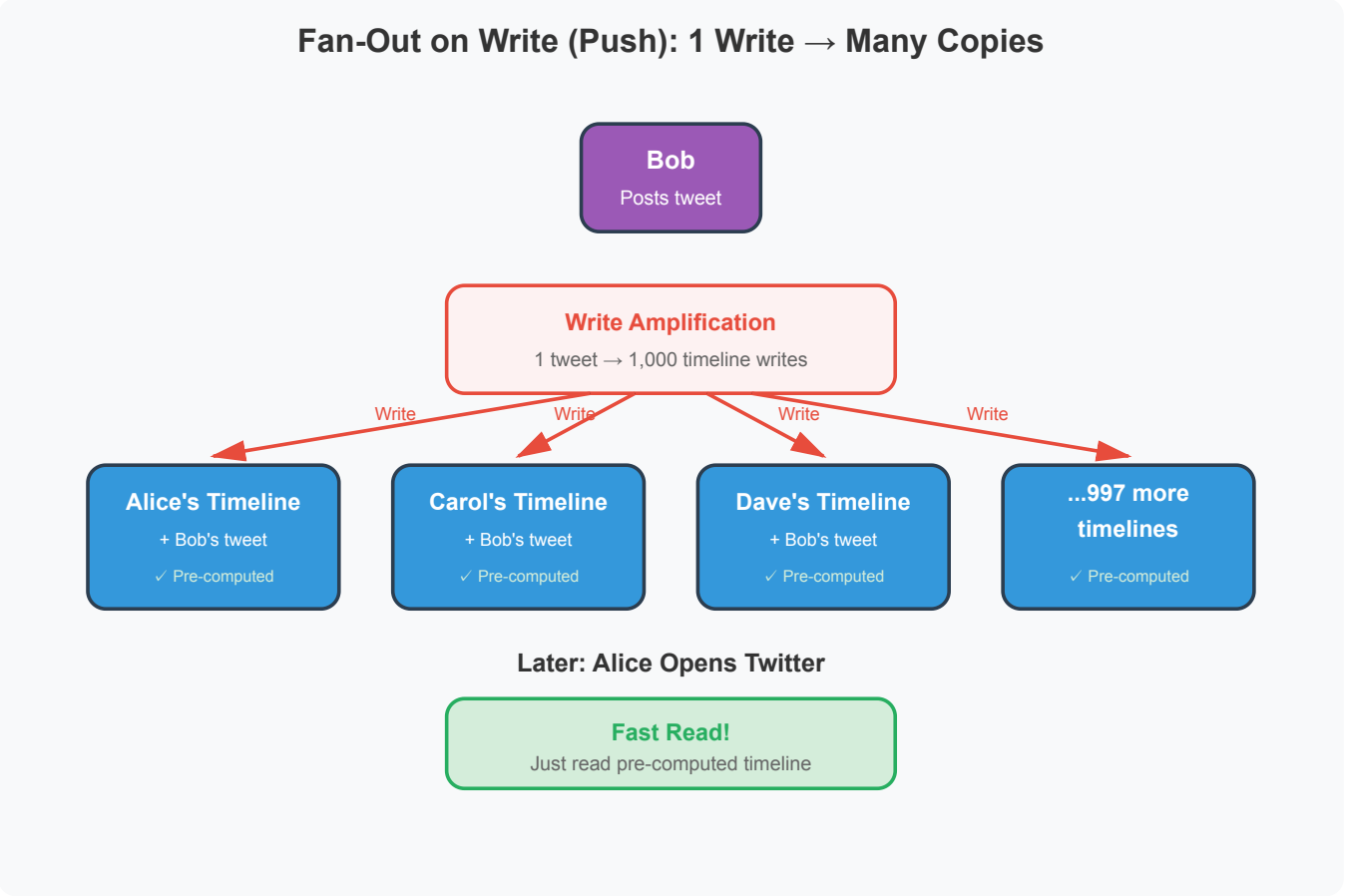
Fan-Out on Write (Push)

When Bob tweets, the system immediately writes that tweet to the timelines of all his followers. Bob has 1,000 followers. The system inserts Bob's tweet into 1,000 pre-computed timelines. When Alice opens Twitter, her timeline is already prepared. Just read and display. No merging. No sorting. Instant load.

This is fan-out on write. One write creates many copies. The timeline becomes a cache. Reading is fast because the work happened during the write. Alice sees Bob's tweet immediately without waiting for queries.

But fan-out on write amplifies writes. A celebrity with 50 million followers posts a tweet. The system must write to 50 million timelines. That single tweet generates 50 million database operations. Popular accounts create massive write spikes. Systems struggle under this amplification.

Storage explodes. Every tweet appears in multiple timelines. A tweet seen by 10,000 users stores 10,000 copies. Disk usage scales with follower count times tweet volume. For accounts with millions of followers, storage costs become prohibitive.



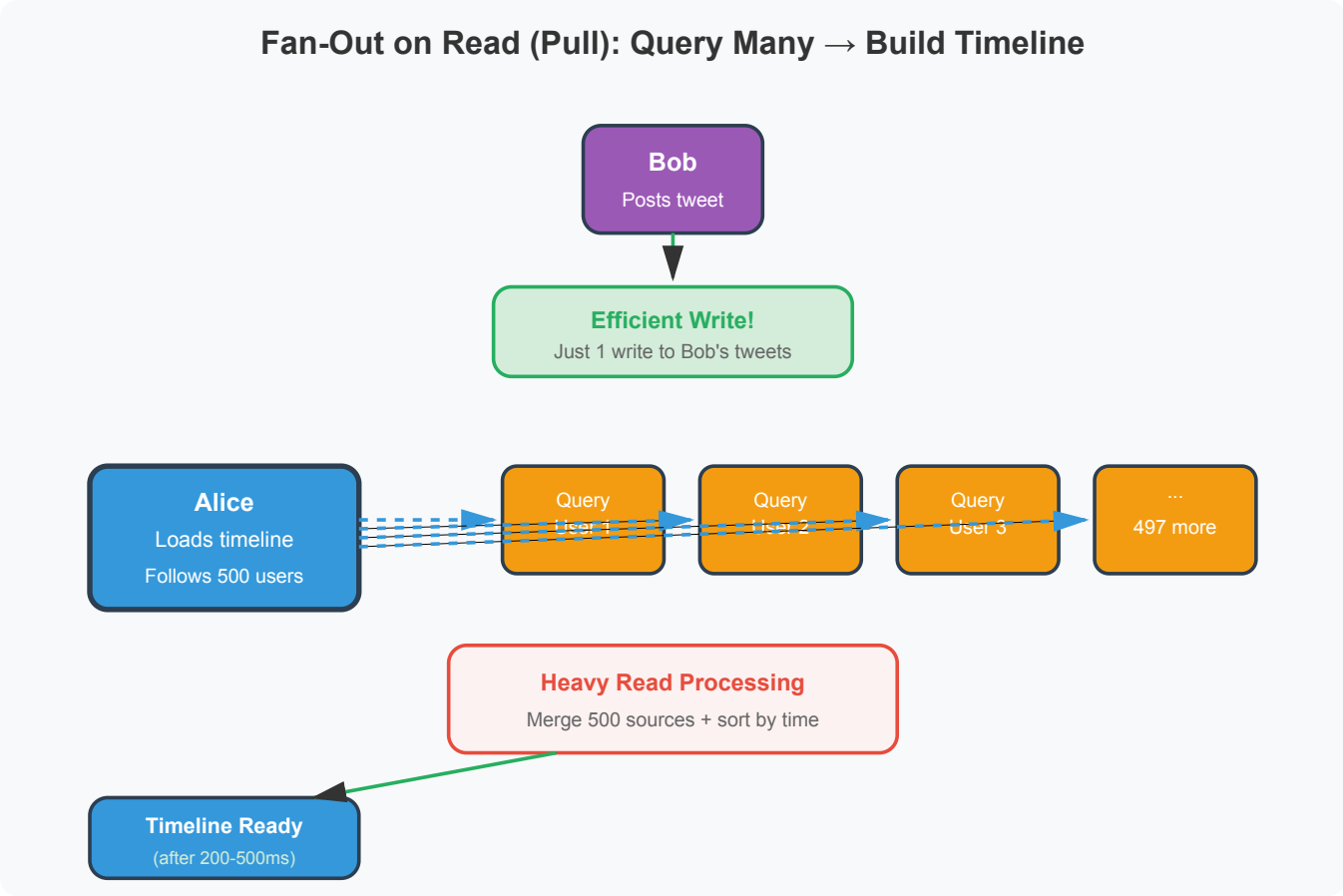
Fan-Out on Read (Pull)

Instead of pre-computing timelines, query on demand. When Alice loads her timeline, fetch recent tweets from all 500 accounts she follows. Merge them. Sort by timestamp. Return the top 50. This is fan-out on read. The work happens during the read instead of the write.

No write amplification exists. Bob tweets once. The system stores one copy. Regardless of follower count, it is one write operation. Celebrities and regular users cost the same on the write path. Storage stays efficient. No duplicate tweets across millions of timelines.

But reads become expensive. Alice loads her timeline. Query 500 accounts. If each account has recent tweets, scan thousands of rows. Merge and sort. This computation happens every time Alice refreshes. Popular users like Alice refresh frequently. Multiply computation across millions of active users.

Latency increases. Querying hundreds of accounts takes time. Merging and sorting adds more. Alice waits several hundred milliseconds for her timeline to load. The UX suffers.

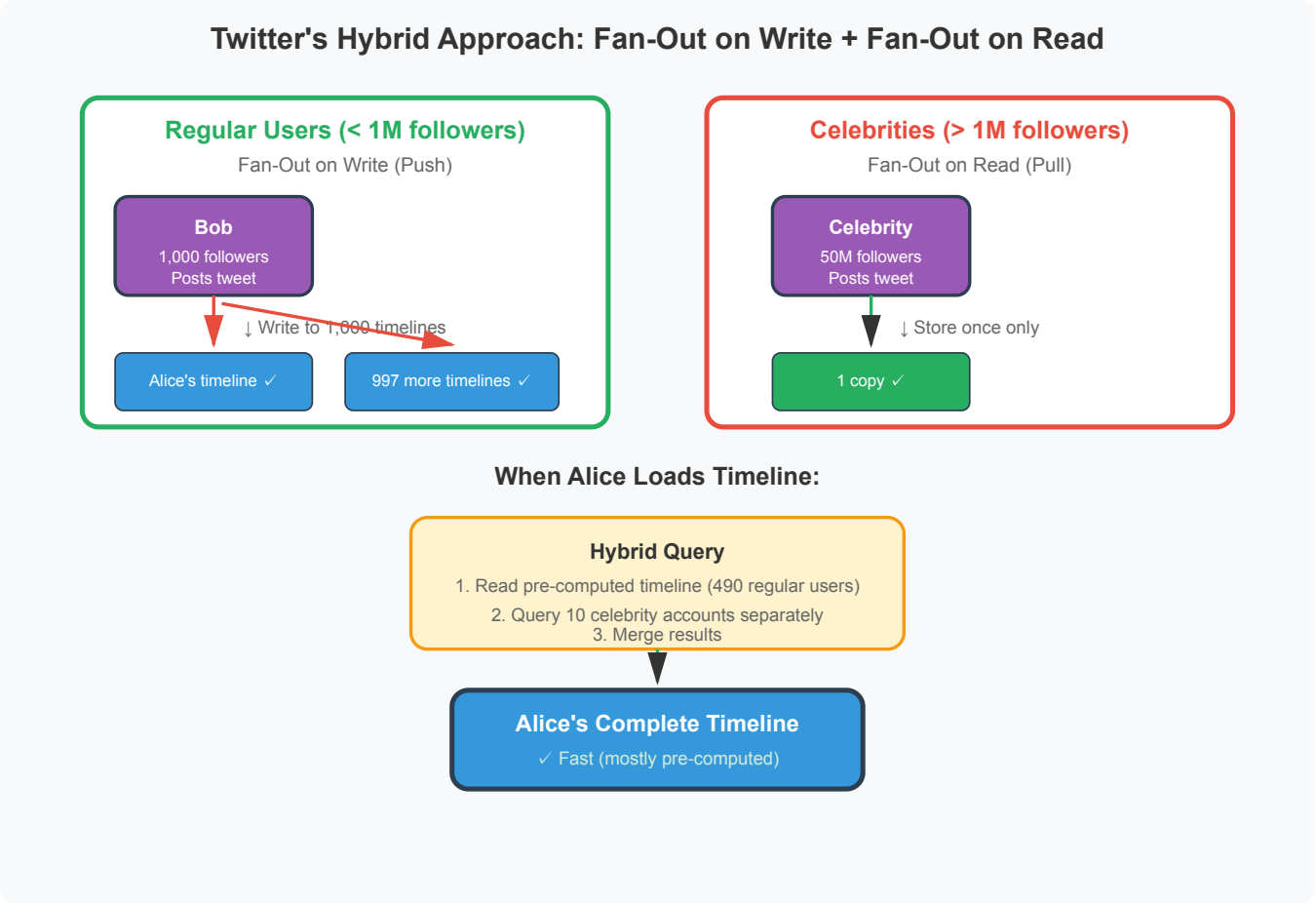


Hybrid Approach

Twitter uses both strategies based on user profiles. Most accounts have modest follower counts. Twitter uses fan-out on write for them. When Bob posts and has 1,000 followers, write to 1,000 timelines. The write amplification is manageable. Alice's timeline loads instantly.

Celebrities have millions of followers. Fan-out on write breaks. Instead, Twitter uses fan-out on read. When a celebrity tweets, store it once. When Alice loads her timeline, include a query for celebrities she follows. Merge celebrity tweets with the pre-computed timeline from regular accounts.

Alice follows 490 regular users and 10 celebrities. The system pre-computes a timeline from the 490 regular users using fan-out on write. When Alice loads her timeline, it queries the 10 celebrity accounts separately and merges those tweets with her pre-computed feed. This hybrid gives Alice fast loads while avoiding write amplification for celebrities.



The threshold for switching strategies varies. Twitter might use 1 million followers as the cutoff. Accounts below use fan-out on write. Accounts above use fan-out on read. The exact threshold balances write cost against read complexity.

Implementation Details

Twitter tracks follower counts. When an account crosses the threshold, switch strategies. New tweets from that account stop writing to follower timelines. Instead, queries fetch those tweets on read. The transition is gradual. Existing timeline entries remain. New tweets follow the new pattern.

During read, the system runs two queries. One fetches the pre-computed timeline. Another queries high-follower accounts. Merge results in memory and return the top items. Caching helps. Cache celebrity tweets for a few seconds. Multiple users requesting timelines share the cached query results.

For inactive users who rarely open Twitter, skip fan-out on write entirely. When they eventually load their timeline, compute it on demand. No point maintaining pre-computed timelines for users who have not logged in for months. Detect activity patterns and adapt.

Trade-Offs

Fan-out on write optimizes reads at the cost of writes. Use it when read frequency exceeds write frequency. Most Twitter users read more than they write. Pre-computing timelines makes sense for the majority.

Fan-out on read optimizes writes at the cost of reads. Use it when writes would amplify excessively. Celebrities post tweets that reach millions. Avoiding that write amplification justifies slower timeline construction.

The hybrid approach combines both. Optimize for the common case (regular users) while handling edge cases (celebrities) differently. This pattern applies beyond Twitter. Any system with highly skewed distribution benefits. E-commerce recommendations. Social networks. Content feeds.

During system design interviews, discuss fan-out strategies. Explain write amplification. Show how follower count affects the decision. Demonstrate awareness of scale implications. This hybrid thinking separates strong candidates from those who only know one pattern.

References

- Original twitter architecture talk by Twitter engineers in 2016
- Design Twitter by System Design School